SymK Product Designer’s Manual
Version: draft — for people who actually ship things

---

0. About This Manual

0.1 What SymK Is

SymK is your **thinking OS** for new products.

It doesn’t sell anything by itself. It gives you:

- A **Project Life Cycle (PLC)** for moving an idea from “hmm…” to “this is worth building”.
- A clear **collaboration model** between:
  - **You** (product designer / owner / architect),
  - and **AI** (your overcaffeinated analyst and editor).
- A set of **templates and folder conventions** so every product uses the same skeleton.

You use SymK when:

- You have an idea and want to know if it’s real or self-deception.
- You need to define a product sharply enough that architecture and engineering can move.
- You want to compare multiple ideas with the same yardstick.

0.2 Who This Manual Is For

This manual is aimed at:

- Product designers and product owners.
- Technical founders and architects who make product decisions.
- Anyone responsible for **defining** what the product is and is not.

It is **not** for:

- People who only want slideware.
- People who worship process for its own sake.

If you use templates to hide lack of thought, SymK will feel hostile. That’s intentional.

0.3 What This Manual Covers

This manual focuses on **new product definition**, using SymK:

- How SymK thinks about products (mindset).
- How the **PLC** structure works (especially PLC_1 — Discovery).
- How to run an idea through SymK in **three passes**:
  1. Sketch
  2. Attack
  3. Decide
- How to hand off a **clean, honest product definition** into Design (PLC_2).

It does **not**:

- Teach basic UX / UI / marketing.

- Replace technical design specs or backlog grooming.
- Guarantee that your idea is good. It helps you find out, fast.

0.4 How to Use This Manual

Recommended flow:

1. Read **Chapter 1** once — get the mindset.
2. Skim **Chapter 2** — understand the PLC map.
3. When you have a new idea:
   - Use **Chapter 3** as your playbook.
4. Use **Chapter 4** to get more out of AI.
5. Use **Chapter 5** as checklists when you're in a hurry.

If you catch yourself "filling fields" instead of thinking, stop. You've gone off-script.

0.5 Key Concepts in One Shot

- **PLC_META**
  The master life cycle defined inside SymK. It says:
  - which phases exist,
  - what each node (1.1, 1.2, …) means,
  - where templates live.

- **Product PLC**
  A specific instance for a product:
  - `PLC_IaaC`, `PLC_LexBrain`, etc.
  - Same structure, filled with product-specific content.

- **AI Role**
  AI is not a ghostwriter. In SymK, AI is used to:
  - draft,
  - attack,
  - compress,
  not just "produce more words".

---

1. SymK Mindset for Product Design

1.1 What We Optimize For

SymK optimizes for three things:

1. **Brutal clarity**
   - What problem, for whom, under which constraints?

2. **Cheap failures**
   - Kill weak ideas early, with a written rationale.

3. **Transferable product definitions**
   - When something survives, you can hand it to architects/engineers without a nine-hour meeting.

Everything else — beauty of documents, formality, buzzwords — is secondary.

1.2 Human vs AI: Clear Roles

1.2.1 You (Product Designer / Owner / Architect)

You are responsible for:

- **Choosing the arena**
  - Which domain, which segment, which constraints matter.
- **Owning decisions**
  - Go / No-Go / Internal-only / Consulting-first / Career-only.
- **Interpreting reality**
  - What is politically possible, what is operationally realistic.

You are not a “prompt operator”. You’re the one who pays if the idea is wrong.

1.2.2 AI (SymK Sidekick)

AI is responsible for:

- **Breadth**
  - Competitors, analogies, edge-cases, alternative framings.
- **Speed**
  - Multiple drafts, rewrites, reorders in minutes.
- **Attack**
  - Pointing out contradictions, missing pieces, hand-waving.

If AI always agrees with you, treat that as a bug.

1.3 What “Good” Looks Like

A good SymK product definition:

- Fits in a **small set of sharp docs**:
  - 1.1 Product Vision
  - 1.2 Market Research (focused)
  - 1.3 Stakeholder Map
  - 1.6 Discovery Outcomes (+ sub-templates)
- Makes a clear call:
  - “This is a **Go** for PLC_2, with constraints X, Y, Z.”
  - or “This is **No-Go**, because A, B, C.”
- Makes it easy to answer:
  - “Why this?”
  - “Why now?”
  - “Why by us?”

If a stranger with domain knowledge can read your 1.x set and understand the bet in under 30 minutes, you’re doing it right.

1.4 SymK Anti-Patterns

Avoid these:

- **Template Worship**
  - Filling templates like tax forms. SymK templates are prompts for thinking, not forms for compliance.

- **Vision Inflation**
  - Turning 1.1 into a science-fiction novella. SymK favors **constrained visions** with clear “not in scope”.

- **Market Fog**
  - 1.2 sounding like generic industry overviews with no segment, no wedge, no opinion.

- **Stakeholder Amnesia**
  - No serious analysis of who can block, sabotage, delay, or accelerate your product.

- **Outcome Cowardice**
  - Avoiding clear outcomes in 1.6. “Let’s explore more” forever = slow-motion No-Go without the courage to say it.

SymK is deliberately designed to make these patterns visible and uncomfortable.

---

2. The SymK PLC Framework (Designer View)

2.1 PLC Overview

The high-level PLC looks like this:

1. **PLC_1 — Discovery**
   Mission: decide if the idea deserves architecture & engineering.

2. **PLC_2 — Architecture & Design**
   Mission: shape the system so it can be built and operated safely.

3. **PLC_3 — Build & Integration**
   Mission: implement, integrate, and automate.

4. **PLC_4 — Validation & Observability**
   Mission: check if reality matches the story.

5. **PLC_5+ — Evolution / Portfolio**
   Mission: manage the product over time and across a portfolio.

As a product designer, you live mostly in **PLC_1** and the **front door of PLC_2**.

2.2 PLC_1 — Discovery as Product Definition Engine

PLC_1 is broken into nodes that already exist in your folder tree:

- **1.1 Product Vision**
  - What problem, for whom, with what ambition and constraints.

- **1.2 Market Research**
  - Segments, alternatives, competitors, and the “why us / why now” logic.

- **1.3 Stakeholder Map**
  - People and institutions who matter:
    - buyers, users, blockers, partners, regulators…

- **1.4 Discovery Methodologies**
  - How you're actually investigating:
    - interviews, desk research, data pulls, experiments.

- **1.5 Discovery Tools**
  - Concrete tools you use:
    - templates, scripts, AI prompts, external services.

- **1.6 Discovery Outcomes**
  - The decision layer:
    - 1.6.1 Decision Framework
    - 1.6.2 Business Size Fit
    - 1.6.3 Consulting Leverage
    - 1.6.4 Career Leverage
    - 1.6.5 Success Metrics & Risks

Your job:

- Use 1.1–1.5 to **think and gather evidence**.
- Use 1.6.x to **decide and document**.

2.3 PLC_2 — Design Bridge (What Designers Need to Prepare)

PLC_2 is where system-level design lives, but product definition **feeds it**.

The core nodes (META-level) are:

- **2.1 System Context & Boundaries**
  - Where this product sits in the ecosystem.
- **2.2 Domain Model & Core Flows**
  - Main entities, main user journeys, key workflows.
- **2.3 Non-Functional Constraints**
  - Security, scalability, compliance, performance, support.
- **2.4 Architecture Options & Trade-Offs**
  - Major patterns considered, with pros/cons.
- **2.5 Chosen Architecture & Rationale**
  - The decision record.

As a product designer, you're not expected to draw every tech detail, but you **must**:

- Make sure 1.x answers are sharp enough that:
  - 2.1–2.3 can be filled without guesswork.
- Capture **non-functional constraints** early:
  - "Must pass compliance X", "Must run air-gapped", "Latency ceiling Y", etc.

If architects have to divine these from your product vision, you've failed the handoff.

2.4 On-Disk Conventions (Why You Should Care)

SymK uses a strict folder scheme:

- `PLC/`
  - `1_Discovery/`
    - `1.1_Product_Vision_Template/`

    - `1.1_Product_Vision.md` (spec: what this node is)
    - `1.1_Product_Vision_Template.md` (blank/guide to fill)
  - `1.2_Market_Research_Template/`
    - `1.2_Market_Research.md`
    - `1.2_Market_Research_Template.md`
  - `…`
  - `1.6_Discovery_Outcomes/`
    - `1.6_Discovery_Outcomes.md`
    - `1.6.1_Decision_Framework_Template/…`
    - …

For each **product**:

- You either:
  - Copy the templates into a `PLC_<ProductName>/` tree, or
  - Keep them in a repo where templates are read-only and product-specific content is stored alongside.

As a designer, this matters because:

- Tools and AI scripts expect this structure.
- It lets you compare PLCs across products without hunting.

2.5 META PLC vs Product PLCs (Designer Impact)

- **META** (in SymK repo):
  - Defines the meaning of each node.
  - Contains the official templates and explanations.

- **Product PLCs**:
  - Are where your actual product definitions live.
  - Must respect node semantics, but can:
    - extend with extra notes,
    - add variant templates if needed.

Net effect for you:

- You never invent structure from scratch.
- You focus on content:
  - “What is true for this product at 1.1, 1.2, 1.3, …?”

---

3. Running a New Product Through SymK

This is the **practical playbook**. Use it with a real idea in hand.

3.1 Before You Start: Define the Seed

Write down, in plain language:

- Working name (doesn’t have to be final).
- One-sentence problem statement.
- Who you believe the primary beneficiary is.
- Why you suspect this is worth your time.

This is not 1.1 yet. This is your **zero page**.

You can even keep it in a file like:

- `PLC_<Product>/0_Seed/0.0_Seed_Notes.md`

3.2 Pass 1 — Sketch (Fast, Imperfect, Complete)

Goal: get a **rough pass** through all 1.x nodes.

Process:

1. Duplicate each `1.x_*_Template.md` into a **product-specific location**.
2. For each:
   - Fill in **short, honest bullets**.
   - Use AI to:
     - tidy text,
     - suggest missing angles,
     - but keep things lightweight.

Guidelines:

- 1.1 Product Vision — two pages max.
- 1.2 Market Research — focus on:
  - segments,
  - substitutes,
  - real “why now”.
- 1.3 Stakeholder Map — don’t list the universe, list the people who can kill or save this.
- 1.4/1.5 — be realistic about how you’ll discover truth (methods) and which tools you’ll actually touch.
- 1.6.x — leave almost empty during Pass 1; just note initial guesses.

Treat Pass 1 as “good enough to argue with”.

3.3 Pass 2 — Attack (With AI and Your Own Skepticism)

Goal: **stress-test** your own story.

Mechanics:

- Ask AI to:
  - Compare 1.1 vs 1.2 vs 1.3 and:
    - find contradictions,
    - call out hand-waving,
    - ask questions you’re avoiding.
  - Propose **alternative lenses**:
    - alternative go-to-market,
    - different primary segment,
    - different “minimum viable slice”.

Concrete prompts you can use (adapted per node):

- “Assume you’re an unfriendly analyst. Attack this Product Vision based on the Market Research and Stakeholder Map.”

- “List the top 10 ways this idea could be smaller, sharper, or more realistic.”
- “Given this 1.2 and 1.3, what’s the most likely way this fails that we’re not discussing?”

Out of this attack pass, you:

- Update 1.1, 1.2, 1.3.
- Refine methods in 1.4.
- Clarify what you **will not do** in this product.

3.4 Pass 3 — Decide (Fill 1.6.x Like You Mean It)

Now you move to **1.6_Discovery_Outcomes** and its sub-templates:

- **1.6.1 Decision Framework**
  - What are the **criteria** for Go / No-Go / Internal / Consulting / Career-only?

- **1.6.2 Business Size Fit**
  - Does this justify productization?
  - Is it:
    - a small but strategic product,
    - a niche expert tool,
    - or just an internal helper?

- **1.6.3 Consulting Leverage**
  - Is this a Trojan horse for high-value consulting?

- **1.6.4 Career Leverage**
  - Does this significantly move your expertise, credibility, position?

- **1.6.5 Success Metrics and Risks**
  - What will you measure?
  - What are the biggest known risks?
  - What are the scary **unknowns**?

Finally: make the call.

SymK treat these outcomes as **equally respectable**:

- **Go $\rightarrow$ PLC_2**
  “We will design and later build this as a product.”

- **No-Go**
  “We documented why this is not worth it. Done.”

- **Internal-only**
  “Worth building, but only as an internal tool.”

- **Consulting-first**
  “Best path is to sell expertise/services, not a standalone product (yet).”

- **Career-only**
  “This is mainly for our own learning, positioning, or portfolio.”

Write the decision clearly at the top of 1.6 and date it.

3.5 If Go: Bridge Into PLC_2 (Design Starter Pack)

If the outcome is **Go** (or “Internal-only” but significant):

- Create a minimal **PLC_2 starter**:

  - `PLC_<Product>/2_Design/`
    - `2.1_System_Context_and_Boundaries.md`
    - `2.2_Domain_Model_and_Core_Flows.md`
    - `2.3_Non_Functional_Constraints.md`
    - `2.4_Architecture_Options_and_Tradeoffs.md`
    - `2.5_Chosen_Architecture_and_Rationale.md`

As a designer, you do **not** need to fill these like an architect would, but you should:

- Draft:
  - high-level system context (2.1),
  - user flows and product slices (2.2),
  - product-driven constraints (2.3: compliance, latency, privacy, etc.).

This is your **handoff bridge** to the technical design phase.

---

4. Using AI Effectively in SymK

4.1 Three Roles for AI

Use AI mainly for:

1. **Drafting**
   - Turn your bullet notes into a readable draft.
   - Keep the structure, don’t let AI “smooth away” important edges.

2. **Attacking**
   - Cross-check nodes:
     - “Read 1.1, 1.2, 1.3 and list inconsistencies.”
   - Ask for “worst case critics”:
     - investors, customers, regulators, operators.

3. **Compressing**
   - Produce:
     - 1-page executive summary of PLC_1.
     - 1-page handoff doc for PLC_2.

4.2 Prompt Patterns per Node

Examples:

- **1.1 Product Vision**
  - “Rewrite this product vision to be:
    - explicit about target segment,
    - crisp about what is out of scope,
    - limited to 1–2 pages.”

- **1.2 Market Research**
  - “Given this vision, identify:
    - competing products,
    - non-obvious substitutes,
    - the most dangerous ‘do nothing’ alternative for the customer.”

- **1.3 Stakeholder Map**
  - “List likely stakeholders by:
    - decision power,
    - daily pain,
    - potential to block.
    Then check if my stakeholder map misses important ones.”

- **1.6 Outcomes**
  - “Given all 1.x docs, argue for:
    - Go,
    - No-Go,
    - Internal-only,
    - Consulting-first.
    Then pick the one with the strongest evidence and explain why.”

4.3 Letting AI Disagree

Don’t ask AI for reassurance. Ask for attack.

Good patterns:

- “Assume you are trying to convince me that this is **not** a productization candidate. Go.”
- “Assume you are a skeptical buyer. What would you ask before paying?”
- “Assume you are future-me, angry that I wasted a year on this. What would I say?”

If the answers don’t make you at least slightly uncomfortable, dig deeper.

---

5. Checklists

5.1 New Product Quick Checklist (PLC_1)

You’re allowed to move on only if:

- [ ] 1.1 is under control: sharp, scoped, with clear non-goals.
- [ ] 1.2 names specific segments and alternatives, not generic “market trends”.
- [ ] 1.3 has real people/roles, including likely blockers.
- [ ] 1.4 states how you will actually learn (not just buzzword methodologies).
- [ ] 1.5 lists real tools you’ll touch (not a laundry list of everything you’ve heard of).
- [ ] 1.6 has a written decision with reasoning, not just “TBD”.

5.2 No-Go Quick Checklist

You may declare **No-Go** with a straight face when:

- [ ] Market is too small or too messy for your strategic goals.
- [ ] There is no clear wedge: existing alternatives solve the problem “good enough”.
- [ ] Key stakeholders will never say yes in the current environment.

- [ ] Non-functional constraints make this product absurdly expensive to build or maintain.
- [ ] The best version of the idea is clearly **consulting** or **career leverage**, not a scalable product.

A clean No-Go is a win. You just avoided months/years of sunk cost.

5.3 Design Handoff Checklist (into PLC_2)

Before handing to architects/engineering:

- [ ] 1.1–1.6 are current and consistent.
- [ ] A minimal 2.1–2.3 exists:
  - [ ] System context draft (2.1).
  - [ ] Core flows and slices draft (2.2).
  - [ ] Non-functional constraints draft (2.3).
- [ ] There is a clear decision note:
  - productization path,
  - initial target segment,
  - constraints that **must not** be violated.

If any of this is missing, expect architecture/design to guess — and then don’t complain.